Relatório Técnico: Desenvolvimento de Biblioteca Modular Avançada de Tópicos Matemáticos

André De Vlieger Lima (16992758)
Lucas Correa Meira (15695588)
Vitor Wallace de Moraes Alves (8610353)

19 de Outubro de 2026

1 Descrição do Problema e do Domínio Escolhido

Em sistemas computacionais de alto desempenho, engines gráficas e simulações científicas, a modularidade e a reusabilidade de código são requisitos indispensáveis. O problema abordado consiste na criação de um ecossistema coeso de ferramentas matemáticas e geométricas na linguagem C, estruturado na forma de uma biblioteca reutilizável seguindo boas práticas de engenharia de software.

O domínio escolhido abrange cinco subáreas inter-relacionadas da matemática aplicada e da computação gráfica:

1. **Álgebra Linear e Operações Vetoriais** (`vetores`): manipulação de vetores estáticos e dinâmicos em 2D, 3D eN dimensões;
2. **Geometria Analítica 2D/3D** (`geometria`): cálculo de áreas, perímetros, centroides e volumes em formas regulares e malhas/polígonos irregulares;
3. **Cálculo Diferencial e Integral** (`calculo`): diferenciação finita, vetores gradientes e integração numérica simples e dupla;
4. **Métodos Numéricos** (`calculo_numerico`): localização de raízes de equações não lineares através de métodos iterativos com tratamento formal de erros;
5. **Polinômios Esparsos** (`polinomios`): representação e manipulação eficiente de polinômios de alto grau utilizando listas encadeadas dinâmicas.

2 Arquitetura do Software

A arquitetura do projeto adota o princípio do encapsulamento estrito e da visibilidade mínima. A estrutura física de diretórios separa os contratos de interface pública, o código de implementação, os testes automatizados e a aplicação cliente final:

```
projeto/
|-- include/           # Interface publica (.h)
|   |-- vetores.h
|   |-- geometria.h
|   |-- calculo.h
|   |-- calculo_numerico.h
|   `-- polinomios.h
|-- src/               # Implementacao interna (.c)
|   |-- vetores.c
|   |-- geometria.c
|   |-- calculo.c
|   |-- calculo_numerico.c
|   `-- polinomios.c
|-- tests/             # Testes automatizados
|   `-- test_main.c
|-- app/               # Aplicacao cliente
|   `-- main.c
|-- build/             # Objetos intermediarios (.o)
|-- lib/               # Bibliotecas geradas (.a, .so)
|-- bin/               # Executaveis finais
|-- Makefile           # Automacao de compilacao
`-- README.md          # Manual de uso
```

3 Organização e Responsabilidade dos Módulos

A biblioteca é composta por 5 módulos funcionalmente distintos:

Tabela 1: Módulos da biblioteca e suas responsabilidades

Módulo	**Arquivos**	**Responsabilidade Funcional**
`vetores`	`vetores.h / .c`	Operações vetoriais, normas, produtos escalar/vetoria
`geometria`	`geometria.h / .c`	Geometria computacional: Shoelace, Ray-Casting e v
`calculo`	`calculo.h / .c`	Diferenciação centrada, gradientes, trapézios e ponto
`calculo_numerico`	`calculo_numerico.h / .c`	Busca de raízes via Bissecção e Newton-Raphson com
`polinomios`	`polinomios.h / .c`	Polinômios esparsos com alocação dinâmica e listas e

4 Descrição da API Pública

A API pública exporta mais de 20 funções através dos arquivos de cabeçalho. Os detalhes de implementação permanecem restritos aos arquivos de código-fonte (`.c`).

4.1 Módulovetores.h

- `VetorND vec_criar_nd(int dimension, const double * components)`: aloca vetor$N$-dimensional dinâmico;
- `void vec_liberar_nd(VetorND * v)`: desaloca a memória do vetor dinâmico;
- `Vetor3D vec_somar3d(Vetor3D v1, Vetor3D v2)`: soma de vetores em$\mathbb{R}^3$;
- `double vec_produto_escalar3d(Vetor3D v1, Vetor3D v2)` e`vec_produto_vetorial3d`: produto escalar e vetorial.

4.2 Módulogeometria.h

- `double geo_area_poligono_irregular(PoligonoIrregular2D p)`: área por Fórmula de Shoelace;
- `int geo_ponto_em_poligono_irregular(Ponto2D ponto, PoligonoIrregular2D p)`: algoritmo de Ray-Casting;
- `double geo_volume_malha_fechada(MalhaIrregular3D m)`: volume via decomposição tetraédrica.

4.3 Móduloscalculo.h ecalculo_numerico.h

- `double calc_derivada_1d(Funcao1D f, double x, double h)`: aproximação por diferenças finitas centradas;
- `void calc_gradiente(FuncaoND f, const double * ponto, int num_vars, double h, double * res)`: vetor gradiente;
- `StatusCalculo num_bisseccao(...)` e`num_newton_raphson(...)`: solução iterativa de raízes com retorno de`StatusCalculo`.

4.4 Módulopolinomios.h

- `Polinomio poli_criar_vazio(void)`: inicializa polinômio nulo;
- `void poli_adicionar_termo(Polinomio * p, double coef, int exp)`: insere termo ordenado por expoente;
- `Polinomio poli_derivada_algebrica(Polinomio p)`: gera a derivada analítica exata.

5 Descrição da Aplicação Cliente

A aplicação cliente, implementada no arquivo app/main.c, tem como objetivo demonstrar a utilização prática e integrada das funcionalidades fornecidas pela biblioteca modular. Ela funciona como um programa executável independente que consome a API pública projetada.

5.1 Respeito Estrito à API Pública

Conforme exigido pelos requisitos de arquitetura e encapsulamento do projeto, a aplicação cliente:

- Apenas os cabeçalhos presentes no diretório [include/] (vetores.h, polinomios.h, geometria.h, calculo.h e calculo_numerico.h) são importados via #include;
- Em nenhuma hipótese o arquivo app/main.c faz inclusão direta de arquivos localizados no diretório [src/];
- Não acessa estruturas internas/privadas: toda a manipulação de dados ocorre por meio das funções públicas expostas e dos tipos definidos na interface pública.

6 Processo de Compilação Utilizando Makefile

O build do projeto é gerido pelo`Makefile`, contendo os seguintes alvos padronizados:

- `make all`: Compila os módulos, gera as bibliotecas estática/dinâmica e o executável cliente;
- `make clean`: Remove todos os artefatos gerados nas pastas`build/`,`lib/` e`bin/`;
- `make run`: Compila se necessário e executa a aplicação;
- `make test`: Compila e executa a suíte de testes automatizados;
- `make install PREFIX=$HOME/.local`: Instala cabeçalhos e bibliotecas no diretório de destino.

7 Estratégia de Compilação Incremental

As dependências são mapeadas de forma que cada objeto em`build/%.o` depende exclusivamente de seu arquivo fonte`src/%.c` e do respectivo cabeçalho em`include/%.h`. Alterar um módulo recompila apenas o arquivo alterado e refaz o link, poupando tempo de compilação.

8 Geração e Utilização da Biblioteca Estática

A biblioteca estática (`libapp.a`) é criada via utilitário`ar`:

```
gcc -Wall -Wextra -Iinclude -c src/vetores.c -o build/vetores.o
gcc -Wall -Wextra -Iinclude -c src/polinomios.c -o build/
   polinomios.o
gcc -Wall -Wextra -Iinclude -c src/geometria.c -o build/geometria.
   o
gcc -Wall -Wextra -Iinclude -c src/calculo.c -o build/calculo.o
gcc -Wall -Wextra -Iinclude -c src/calculo_numerico.c -o build/
   calculo_numerico.o
ar rcs lib/libapp.a build/*.o
gcc -Iinclude app/main.c lib/libapp.a -lm -o bin/app_estatico

./bin/app_estatico
```

9 Geração e Utilização da Biblioteca Dinâmica

A biblioteca dinâmica (`libapp.so`) exige a opção`-fPIC` e a flag`-shared`:

```
gcc -Wall -Wextra -fPIC -Iinclude -c src/*.c
# Move os .o gerados para a pasta build (opcional, dependendo de
   como organiza)
mv *.o build/
gcc -shared build/*.o -o lib/libapp.so
gcc -Iinclude app/main.c -Llib -lapp -lm -o bin/app_dinamico
LD_LIBRARY_PATH=lib ./bin/app_dinamico
```

10 Comparação entre Ligação Estática e Dinâmica

Este projeto implementa e analisa as duas abordagens fundamentais: ligação estática (libapp.a) e ligação dinâmica (libapp.so).

10.1 Biblioteca Estática (libapp.a)

A biblioteca estática é uma coleção de arquivos objeto (.o) empacotados em um único arquivo de arquivo, utilizando o utilitário*ar*.

- Mecanismo: durante o processo de compilação, o linker extrai os símbolos necessários da biblioteca e os copia diretamente para dentro do binário executável final;
- Independência em Runtime: o executável resultante é totalmente autônomo. Ele pode ser movido para outro ambiente Linux idêntico e executado sem a necessidade de manter o arquivo libapp.a no sistema.

10.2 Biblioteca Dinâmica (libapp.so)

A biblioteca dinâmica (Shared Object) é gerada compilando o código-fonte com a flag de código independente de posição (-fPIC) e vinculada via gcc -shared.

- Mecanismo: O linker não copia o código da biblioteca para o executável; em vez disso, insere tabelas de referência a símbolos externos. A resolução real dos endereços das funções é delegada ao carregador do sistema operacional (dynamic linker) no momento em que o programa é iniciado;
- Dependência em Runtime: O executável necessita que o arquivo .so esteja presente no disco e localizado em diretórios conhecidos pelo sistema ou especificados pela variável de ambiente LD_LIBRARY_PATH.

10.3 Resultados e Demonstração Prática no Projeto

Durante o desenvolvimento do trabalho, as seguintes evidências experimentais comprovaram as diferenças teóricas:

10.3.1 Inspeção de Dependências com ldd:

Ao executar o comando ldd bin/client na versão com ligação dinâmica, o sistema listou explicitamente a dependência de tempo de execução:

1. libapp.so => lib/libapp.so (0x00007f...)
2. libm.so.6 => /lib/x86_64-linux-gnu/libm.so.6 (0x00007f...)
3. libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f...)

10.3.2 O "Teste da Remoção" de Diretório:

Executável Dinâmico: Ao remover ou mover temporariamente o diretório lib e tentar executar ./bin/client, o sistema operacional interrompeu a execução imediatamente exibindo o erro: error while loading shared libraries: libapp.so: cannot open shared object file.

Executável Estático: Ao compilar um binário estático via: gcc -Iinclude app/main.c lib/libapp.a -lm -o bin/client_estatico e em seguida remover a pasta lib/, o programa executou perfeitamente sem falhar, comprovando que todo o código da biblioteca já estava incorporado ao executável.

11 Estratégia e Resultados dos Testes

11.1 Estratégia de Teste

A estratégia adotada apoia-se em três pilares principais:

1. Isolamento Modular: cada módulo da biblioteca possui um arquivo de teste dedicado (test_vetores.c, test_polinomios.c, test_geometria.c, test_calculo.c e test_calculo_num. O orquestrador central test_main.c executa sequencialmente as suítes de cada módulo;

2. Validação por Asserções (assert.h): utilizam-se asserções em C para verificar condições lógicas de contorno, estados de erro e propriedades matemáticas esperadas;

3. Comparação com Tolerância Numérica: devido às imprecisões inerentes à representação de ponto flutuante em computadores, operações numéricas não utilizam igualdade estrita (==). Em vez disso, implementou-se a verificação com uma tolerância $=10^{-4}$ via função fabs().

11.2 Execução Automatizada via Makefile

A automação é integrada ao processo de construção do projeto através do alvo make test. O comando compila os arquivos de teste vinculando-os à biblioteca estática e à biblioteca matemática (-Llib -lapp -lm) e executa o binário resultante bin/test.

12 Exemplo de Teste que Inicialmente Falhou e Posterior Correção

Em conformidade com as boas práticas de desenvolvimento orientado a testes (TDD) e os requisitos do edital do projeto, registra-se nesta seção um caso real de falha identificada durante a execução da suíte de testes e o respectivo processo de correção.

12.1 Descrição do Erro Encontrado

Durante a execução inicial da suíte de testes do Módulo de Cálculo Diferencial e Integral, a asserção responsável por validar o vetor gradiente da função f (x, y) = x2 + y2 no ponto (1.0, 1.0) falhou. A derivada parcial esperada em relação a x no ponto x = 1.0 é fx(1, 1) = 2 · (1.0) = 2.0.

No entanto, o algoritmo retornou o valor −2.0, ocasionando a interrupção da suíte pelo mecanismo de asserção.

12.2 Saída do Terminal Acusando a Falha

Ao executar o comando make test, a suíte apresentou a seguinte mensagem de erro no terminal:

```
--- Executando Suíte de Testes ---
[MÓDULO] Vetores...
Aprovado: Vetores 3D e N-D
[MÓDULO] Polinômios Esparsos...
Aprovado: Polinômios
[MÓDULO] Geometria Analítica 2D/3D...
```

```
Aprovado: Polígonos 2D e Geometria Espacial
[MÓDULO] Cálculo Diferencial e Integral...
test: tests/test_calculo.c:26: testar_calculo: Assertion 'fabs((
    grad[0]) - (2.0)) < (1e-
4)' failed.
make: *** [Makefile:41: test] Abortado (arquivo core criado)
```

12.3 Causa Raiz da Falha

Ao inspecionar o arquivo src/calculo.c, identificou-se um erro de sinal na implementação da aproximação por diferenças finitas centradas na função calc_derivada_parcial:

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$$

No código original, a ordem dos termos no numerador estava invertida, calculando $f(x-h) - f(x+h)$, o que resultava no sinal oposto da derivada:

```c
/* Código INCORRETO em src/calculo.c */
p[var_index] = ponto[var_index] + h;
double l_superior = f(p_sup, num_vars);

p[var_index] = ponto[var_index] - h;
double l_inferior = f(p_inf, num_vars);

// return (f(p,num_vars) - f(p,num_vars))/(2*h); por conta da
    alocação dinamica e do free fui obrigado copiar os valores pra
    f_mais e f_menos
    free(p);
return (l_inferior - l_superior)/(2.0 * h); // ERRO: Ordem
    invertida!
```

12.4 Correção Aplicada

A função foi corrigida no arquivo src/calculo.c, ajustando-se a subtração para a ordem correta$f(x-h) - f(x+h)$:

```c
/* Código CORRIGIDO em src/calculo.c */
p[var_index] = ponto[var_index] + h;
double l_superior = f(p_sup, num_vars);

p[var_index] = ponto[var_index] - h;
double l_inferior = f(p_inf, num_vars);

// return (f(p,num_vars) - f(p,num_vars))/(2*h); por conta da
    alocação dinamica e do free fui obrigado copiar os valores pra
    f_mais e f_menos
    free(p);
return (l_superior - l_inferior)/(2.0 * h); // CORRETO
```

12.5 Validação Pós-Correção

Após recompiar a biblioteca e reexecutar a suíte de testes com make clean && make test, todos os testes passaram com 100% de sucesso:

```
--- Executando Suíte de Testes ---
[MÓDULO] Vetores...
Aprovado: Vetores 3D e N-D
[MÓDULO] Polinômios Esparsos...
Aprovado: Polinômios
[MÓDULO] Geometria Analítica 2D/3D...
Aprovado: Polígonos 2D e Geometria Espacial
[MÓDULO] Cálculo Diferencial e Integral...
Aprovado: Derivadas, Gradientes e Integrais 1D/2D
[MÓDULO] Cálculo Numérico...
Aprovado: Bissecção e Newton-Raphson

Todos os testes passaram com sucesso!
```

13 Estratégia de Versionamento Utilizando Git

A estratégia de versionamento adotada visou manter a integridade da base de código, garantir a rastreabilidade das alterações e documentar a evolução modular da biblioteca, seguindo sempre as melhores práticas de*Atomicidade dos Commits*,*Mensagens de Commit Claras* e*Uso do .gitignore*.

14 Descrição de Branch, Merge e Tag v1.0

Decidiu-se seguir da proposta: cada módulo teria de ser uma Branch separada; dessa forma, poderia-se facilmente delimitar as tarefas do projeto e analisar a questão de falhas no código, já que cada branch teve de se encapsular num pull request que, em fim, lida a um merge.

Seguindo as boas práticas, depois do desenvolvimento do projeto, colocou uma tag v1.0 para o github do mesmo, indicando que o desenvolvimento havia se encerado.

15 Dificuldades Encontradas e Decisões de Projeto

O desenvolvimento da biblioteca modular exigiu decisões cuidadosas de arquitetura para harmonizar desempenho, clareza de código e encapsulamento em linguagem C.

15.1 Dificuldades e Soluções

1. Resolução de Caminho da Biblioteca Dinâmica em Runtime (LD_LIBRARY_PATH):

(a) Problema: ao executar o cliente compilado com ligação dinâmica (./bin/client), o carregador do sistema operacional falhava ao localizar a biblioteca libapp.so;

(b) Decisão de Projeto: para fins de testes e execução rápida, utilizou-se a variável de ambiente export LD_LIBRARY_PATH=./lib:$LD_LIBRARY_PATH. Para o alvo automatizado make run, optou-se por embutir o caminho de busca em tempo de execução via flag do linker (-Wl,-rpath=./lib), permitindo a execução imediata do programa pelo usuário.

2. Gerenciamento de Memória em Polinômios Esparsos e Vetores N-D:

   (a) Problema: risco de vazamento de memória (memory leak) ao manipular listas encadeadas dinâmicas e vetores N -dimensionais alocados com malloc;

   (b) Decisão de Projeto: estabeleceu-se o princípio da responsabilidade explícita na API pública, onde cada função construtora (vec_criar_nd, poli_criar_vazio) obrigatoriamente possui uma função correspondente de destruição/liberação (vec_liberar_nd, poli_liberar), devidamente testadas com verificações de ponteiros nulos.

3. Comparação de Valores de Ponto Flutuante nos Testes

   (a) Problema: comparações diretas de igualdade (==) entre valores do tipo double falhavam devido ao arredondamento;

   (b) Decisão de Projeto: foi adotado a padronização de testes numéricos utilizando uma margem de tolerância $=10^{-4}$ via macro ASSERT_NEAR, assegurando a robustez das asserções.

4. Inversão de Sinal na Derivada Numérica:

   (a) Problema: conforme detalhado na Seção 12, a ordem dos operandos no cálculo da diferença finita estava invertida, resultando em gradientes com sinal trocado;

   (b) Decisão de Projeto: a correção foi guiada pela suíte de testes unitários automatizados, comprovando a eficácia do uso de testes durante o ciclo de desenvolvimento.

16 Resultados Obtidos e Possíveis Melhorias

16.1 Resultados Obtidos

O projeto atingiu plenamente todos os requisitos funcionais, não funcionais e de arquitetura preestabelecida:

- Arquitetura Modular Completa: desenvolvimento bem-sucedido dos 5 módulos matemáticos (vetores, polinomios, geometria, calculo e calculo_numerico) com estrito encapsulamento e separação entre interface (.h) e implementação (.c);

- Geração Dupla de Bibliotecas: automação completa no Makefile para compilação e empacotamento da biblioteca estática (libapp.a) e biblioteca dinâmica (libapp.so);
- Aplicação Cliente Funcional: implementação da aplicação app/main.c que consome exclusivamente a API pública dos módulos e valida o funcionamento integrado da biblioteca;
- Testes Automatizada com 100% de Aprovação: verificação rigorosa de todos os métodos numéricos e geométricos via make test;
- Versionamento Rigoroso: histórico organizado com uso de branches, merges explícitos e identificação da versão final via tag v1.0.

16.2 Possíveis Melhorias Futuras

Como perspectivas de evolução para versões futuras do projeto, destacam-se:

1. Paralelização de Cálculos Intensivos: utilização da biblioteca OpenMP ou POSIX Threads para paralelizar o cálculo de integrais duplas e gradientes N-dimensionais, reduzindo o tempo de processamento em CPUs multicore;
2. Implementação de Métodos Numéricos de Ordem Superior: expansão do módulo de cálculo numérico com métodos de maior convergência;
3. Geração Automática de Documentação: adoção do padrão de comentários Doxygen nos arquivos de cabeçalho (.h) para possibilitar a geração automática de manuais de referência em HTML e PDF;
4. Suíte de Benchmark de Desempenho: adição de um módulo de medição de tempo de execução e perfilamento de memória (profiling) para comparar a eficiência do código em diferentes níveis de otimização do compilador (-O2, -O3).